

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ВІННИЦЬКИЙ НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ
Факультет інтелектуальних інформаційних технологій та автоматизації
Кафедра автоматизації та інтелектуальних інформаційних технологій

**Мова програмування «Трипілля»:
проєктування, архітектура та реалізація інтерпретатора**

Концептуальна робота

Виконав:
ст. гр. 2АКІТР-256
Чернега С. А.

м. Вінниця, 2026

АНОТАЦІЯ

У даній концептуальній роботі представлено проектування та реалізацію інтерпретатора навчальної мови програмування «Трипілля» мовою C++17. Розроблено всі ключові компоненти сучасного інтерпретатора: лексичний аналізатор (токенізатор), синтаксичний аналізатор (парсер), абстрактне синтаксичне дерево (AST), семантичний аналізатор із перевіркою областей видимості, генератор коду (транспіляція в C++) та виконувач (деревний інтерпретатор).

Мова «Трипілля» підтримує базові конструкції програмування: змінні, арифметичні та логічні операції, умовні інструкції, цикли, функції з замиканнями та об'єктно-орієнтоване програмування (класи, методи, інстанціювання). Система типів побудована на `std::variant` з підтримкою чисел, рядків, логічних значень, функцій та об'єктів.

Архітектура інтерпретатора є модульною: кожен етап обробки коду є незалежним компонентом із чітко визначеним інтерфейсом. Для обходу AST застосовано шаблон проектування Visitor, що дозволяє відокремити алгоритми обробки від структури дерева.

Ключові слова: мова програмування, інтерпретатор, C++17, лексичний аналіз, синтаксичний аналіз, абстрактне синтаксичне дерево, семантичний аналіз, транспіляція.

Зміст

1	Вступ	3
1.1	Мета роботи	3
1.2	Завдання	3
2	Огляд архітектури	4
3	Лексичний аналізатор (Lexer)	4
3.1	Типи токенів	4
3.2	Реалізація	4
4	Синтаксичний аналізатор (Parser)	5
4.1	Граматика виразів	5
4.2	Граматика інструкцій	6
4.3	Обробка помилок	6
5	Абстрактне синтаксичне дерево (AST)	7
5.1	Типи вузлів	7
5.2	Visitor pattern	7
6	Семантичний аналізатор (SemanticAnalyzer)	8
6.1	Таблиця символів	8
6.2	Перевірки	8
7	Генератор коду (CodeGenerator)	9
7.1	Приклад генерації	9
7.2	Особливості реалізації	9
8	Інтерпретатор (Interpreter)	10
8.1	Система типів	10
8.2	Оточення (Environment)	10
8.3	Функції та замикання	10
8.4	Класи та об'єкти	11
9	Висновки	11

1 Вступ

Сучасна комп'ютерна інженерія неможлива без глибокого розуміння того, як працюють мови програмування. Розробка власної мови програмування є комплексним завданням, що охоплює лексичний аналіз, синтаксичний розбір, семантичний аналіз, генерацію коду та безпосереднє виконання. Дана робота присвячена проєктуванню та реалізації інтерпретатора для навчальної мови програмування «Трипілля».

Мова отримала назву на честь Трипільської культури — однієї з найдавніших землеробських культур на території сучасної України. Це символізує поєднання давніх традицій із сучасними технологіями.

1.1 Мета роботи

Метою роботи є проєктування та реалізація інтерпретатора власної мови програмування з нуля, використовуючи мову C++17. Основний фокус зроблено на архітектурну чистоту, модульність та розуміння кожного етапу обробки вихідного коду.

1.2 Завдання

1. Спроєктувати синтаксис та семантику мови програмування.
2. Реалізувати лексичний аналізатор (токенізатор).
3. Реалізувати синтаксичний аналізатор (парсер) методом рекурсивного спуску.
4. Побудувати абстрактне синтаксичне дерево (AST).
5. Реалізувати семантичний аналізатор із перевіркою областей видимості.
6. Реалізувати генератор коду (транспіляція в C++).
7. Реалізувати деревний інтерпретатор.
8. Протестувати працездатність усіх компонентів.

2 Огляд архітектури

На рис. 1 зображено загальну архітектуру інтерпретатора мови «Трипілля», яка складається з послідовних етапів обробки вхідного коду:

Код → Лексер → Парсер → AST → Семантичний
аналізатор → Генератор коду / Інтерпретатор

Рисунок 1 — Етапи обробки коду

Кожен етап є незалежним модулем, що взаємодіє з наступним через чітко визначений інтерфейс. Такий підхід забезпечує модульність, тестованість та можливість заміни будь-якого компонента без впливу на решту системи.

3 Лексичний аналізатор (Lexer)

Лексичний аналізатор [1] є першим етапом обробки вихідного коду. Його задача — перетворити послідовність символів у послідовність токенів — значущих лексичних одиниць.

3.1 Типи токенів

Визначено такі категорії токенів:

- **Односимвольні:** дужки, кома, крапка, крапка з комою.
- **Оператори:** арифметичні (+, -, *, /), порівняння (<, >, <=, >=), логічні (!), присвоєння (=), перевірка рівності (==, !=).
- **Літерали:** ідентифікатори, числа (з десятковою частиною), рядки.
- **Ключові слова:** class, fn, let, virtual, override, print, if, else, while.

3.2 Реалізація

Лексер реалізовано як клас `Lexer`, що отримує вихідний код у вигляді рядка та надає метод `nextToken()` для поточного отримання токенів. Внутрішній стан зберігає поточну позицію в коді та номер рядка для звітування про помилки. У лістингу 1 наведено приклад використання лексера.

Лістинг 1 — Приклад роботи лексера

```
1 Lexer lexer("let x = 42;");  
2 Token t;  
3 while ((t = lexer.nextToken()).type != TokenType::  
    ↪ END_OF_FILE) {  
4     // Обробка токена  
5 }
```

Підтримується пропуск пробільних символів та коментарів у стилі C++ (//).

4 Синтаксичний аналізатор (Parser)

Парсер реалізовано методом рекурсивного спуску — це техніка, за якої кожне граматичне правило відображається у відповідну функцію. Такий підхід забезпечує зрозумілість коду та легкість розширення граматики.

4.1 Граматика виразів

Граматика виразів побудована за стандартною ієрархією з урахуванням пріоритетів операторів:

1. Первинні вирази: літерали, змінні, груповані вирази.
2. Виклики функцій.
3. Унарні операції.
4. Мультиплікативні операції (*, /).
5. Адитивні операції (+, -).
6. Операції порівняння.
7. Операції рівності.
8. Присвоєння.

4.2 Граматика інструкцій

Підтримуються такі типи інструкцій:

- `print <вираз>;` — виведення значення.
- `if (<умова>) <інструкція> [else <інструкція>]` — умовне виконання.
- `while (<умова>) <інструкція>` — цикл.
- `{ <інструкції> }` — блок інструкцій (нова область видимості).

4.3 Обробка помилок

Парсер містить механізм синхронізації після синтаксичних помилок. При виявленні помилки парсер пропускає токени до наступної межі інструкції (крапка з комою або ключове слово), що дозволяє виявити кілька помилок за одну спробу розбору. Реалізацію методу синхронізації показано в лістингу 2.

Лістинг 2 — Метод синхронізації

```
1 void Parser::synchronize() {
2     while (currentToken.type != TokenType::END_OF_FILE)
3         ↪ {
4             if (currentToken.type == TokenType::SEMICOLON)
5                 ↪ return;
6             switch (currentToken.type) {
7                 case TokenType::CLASS:
8                 case TokenType::FN:
9                 case TokenType::LET:
10                case TokenType::IF:
11                case TokenType::WHILE:
12                case TokenType::PRINT: return;
13                default: advance();
14            }
15        }
16 }
```

5 Абстрактне синтаксичне дерево (AST)

AST є центральним компонентом інтерпретатора. Воно представляє структуру програми у вигляді дерева вузлів, де кожен вузол відповідає певній конструкції мови.

5.1 Типи вузлів

Ієрархія вузлів AST включає:

- **Вузли виразів:** `BinaryExpr` (бінарні операції), `LiteralExpr` (літерали), `VariableExpr` (змінні), `AssignExpr` (присвоєння), `CallExpr` (виклики).
- **Вузли інструкцій:** `ExpressionStmt` (інструкція-вираз), `PrintStmt` (виведення), `VarStmt` (оголошення змінної), `BlockStmt` (блок), `IfStmt` (умова), `WhileStmt` (цикл).
- **Вузли оголошень:** `FunctionNode` (функція), `ClassNode` (клас).

5.2 Visitor pattern

Для обходу AST використано шаблон проектування `Visitor`. Це дозволяє відокремити алгоритми обробки дерева від структури самих вузлів. Кожен прохід (семантичний аналіз, генерація коду, інтерпретація) реалізує свій власний `ASTVisitor` (див. лістинг 3).

Лістинг 3 — Базовий клас `ASTVisitor`

```
1 class ASTVisitor {  
2 public:  
3     virtual void visit(ProgramNode* node) = 0;  
4     virtual void visit(BinaryExpr* node) = 0;  
5     virtual void visit(LiteralExpr* node) = 0;  
6     virtual void visit(VariableExpr* node) = 0;  
7     // ... решта вузлів  
8 };
```


6 Семантичний аналізатор (SemanticAnalyzer)

Семантичний аналіз перевіряє коректність програми на рівні, що виходить за межі синтаксису.

6.1 Таблиця символів

Для відстеження областей видимості реалізовано клас `SymbolTable`, що утворює ієрархічний ланцюжок:

- Кожен блок (`BlockStmt`) створює нову область видимості.
- Функції та класи також створюють власні області.
- Пошук змінної відбувається від поточної області до глобальної.

6.2 Перевірки

Семантичний аналізатор виконує такі перевірки:

- Чи оголошена змінна перед використанням.
- Чи не відбувається повторного оголошення в тій самій області видимості.
- Чи не виконується присвоєння константі (`isConst`).

У лістингу 4 наведено приклад семантичної перевірки змінної.

Лістинг 4 — Семантична перевірка змінної

```
1 void visit(VariableExpr* node) override {
2     Symbol* symbol = currentScope->resolve(node->name.
  ↪ lexeme);
3     if (!symbol) {
4         ErrorHandler::reportError(
5             "Undefined variable '" + node->name.lexeme
  ↪ + "'");
6     };
7 }
8 }
```

7 Генератор коду (CodeGenerator)

Генератор коду виконує транспіляцію програм мови «Трипілля» у вихідний код C++. Це дозволяє використовувати компілятор C++ для подальшої оптимізації та компіляції в машинний код.

7.1 Приклад генерації

У лістингу 5 наведено приклад вхідної програми мовою «Трипілля».

Лістинг 5 — Вхідна програма мовою «Трипілля»

```
1 fn greet(name) {  
2     print "Hello, " + name;  
3 }  
4  
5 greet("World");
```

Лістинг 6 містить відповідний згенерований C++ код.

Лістинг 6 — Згенерований C++ код

```
1 auto greet(auto name) {  
2     std::cout << "Hello, " + name << std::endl;  
3 }  
4  
5 int main() {  
6     greet("World");  
7     return 0;  
8 }
```

7.2 Особливості реалізації

- Для типів використовується `auto` (виведення типів компілятором C++).
- Інструкція `print` транспілюється в `std::cout << ... << std::endl`.
- Класи відображаються в класи C++, методи — у методи класу.

8 Інтерпретатор (Interpreter)

Інтерпретатор виконує програму безпосередньо, обходячи AST за допомогою шаблону Visitor.

8.1 Система типів

Для представлення значень під час виконання використано `std::variant` (див. лістинг 7):

Лістинг 7 — Тип Value

```
1 using Value = std::variant<
2     std::nullptr_t,           // nil
3     bool,                     // логічне
4     double,                   // число
5     std::string,              // рядок
6     std::shared_ptr<Function>, // функція
7     std::shared_ptr<Class>,    // клас
8     std::shared_ptr<Instance> // екземпляр класу
9 >;
```

8.2 Оточення (Environment)

Для зберігання змінних використовується клас `Environment`, що підтримує:

- Вкладені області видимості (scope chaining).
- Пошук змінної в поточному та зовнішніх оточеннях.
- Присвоєння значень існуючим змінним.

8.3 Функції та замикання

Функції в мові «Трипілля» є об'єктами першого класу. Кожна функція запам'ятовує оточення, в якому була оголошена (замикання), що дозволяє отримувати доступ до зовнішніх змінних під час виконання.

8.4 Класи та об'єкти

Підтримується об'єктно-орієнтоване програмування:

- Оголошення класів та створення екземплярів.
- Методи класу (зокрема конструктор `init`).
- Поля екземпляра, які можна динамічно додавати.

У лістингу 8 наведено приклад програми з класами.

Лістинг 8 — Приклад програми з класами

```
1 class Animal {
2     fn init(name) {
3         this.name = name;
4     }
5
6     fn speak() {
7         print this.name;
8     }
9 }
10
11 let animal = Animal("Dog");
12 animal.speak();
```

9 Висновки

У ході виконання даної концептуальної роботи було:

1. Спроектовано та реалізовано повноцінний інтерпретатор мови програмування «Трипілля» мовою C++17.
2. Реалізовано всі ключові компоненти: лексичний аналізатор, синтаксичний аналізатор, абстрактне синтаксичне дерево, семантичний аналізатор, генератор коду та виконувач.
3. Застосовано сучасні практики програмування: шаблон Visitor, variant-based типізація, автоматичне керування пам'яттю через `shared_ptr`.

4. Архітектуру побудовано модульно, що дозволяє легко розширювати мову новими конструкціями.

Розробка власної мови програмування дає глибоке розуміння принципів роботи компіляторів та інтерпретаторів, що є важливою складовою підготовки фахівця з комп'ютерної інженерії.

Література

- [1] Nystrom R. Crafting Interpreters. — Genever Benning, 2021. — 820 p.
- [2] Ахо А., Лам М., Сеті Р., Ульман Дж. Компілятори: принципи, технології та інструментарій. — 2-е вид. — Вільямс, 2008. — 1184 с.
- [3] Страуструп Б. Мова програмування C++. — 4-е вид. — Вільямс, 2015. — 1328 с.